

LAB-4: Dependency Isolation using Fake and Stub

Situation

Imagine your system is already running. You have a `StudentService` that searches for students, validates data, or calculates averages. However, in a real project a problem appears: another team has not finished their module yet, an API sometimes fails, or the implementation may change in the future.

Question: Should your business logic wait for other modules? Can you continue development independently?

Goal

Extend the LAB-3 architecture so that the system is isolated from concrete implementations and allows control over dependency behavior through interfaces.

General Requirements

- `Program.cs` works only with interfaces
- No business logic in `Main` method
- No class creates its own dependency using `'new'`
- Dependencies are passed via constructor injection
- Fake and Stub implementations are created
- At least two logical branches are demonstrated
- README explains what was isolated and why

PART 1 – Dependency Isolation (Refactor)

Choose one of your LAB-3 strategies and check whether the class depends on a concrete implementation.

```
// Bad example
private IStudentFinder _finder = new FindById();
```

If you find such code, refactor it so that the dependency is passed through the constructor.

PART 2 – Fake (Imitating an Unfinished Module)

Create a Fake class that allows you to work independently from the real implementation. It must implement the same interface and return a stable result.

```
public class FakeStudentFinder : IStudentFinder
{
    public Student? Find(Group g, string query)
    {
        return new Student(1, "Fake", "fake@test.com");
    }
}
```

PART 3 – Stub (Controlled Behavior)

Create a Stub class that allows controlling the result via the constructor and demonstrating different branches of business logic.

```
public class StubStudentFinder : IStudentFinder
{
    private readonly Student? _result;

    public StubStudentFinder(Student? result)
    {
        _result = result;
    }

    public Student? Find(Group g, string query)
    {
        return _result;
    }
}
```

PART 4 – Business Logic Stability

Replace Fake with Stub and change the Stub parameter. The business logic class must not be modified. Only the injected implementation changes.

PART 5 (Challenge) – Runtime Selection

Allow runtime selection between real implementation, Fake, or Stub. Briefly explain in README why such architecture is beneficial in larger systems.

Common Mistakes

- Using 'new' inside the class instead of constructor injection - Using Fake and Stub without a clear purpose - Keeping business logic inside Main method - Modifying StudentService instead of replacing dependency - No architectural explanation in README

Evaluation (10 points)

2 points – Correct dependency isolation 2 points – Fake implementation 2 points – Stub implementation 2 points – Clear demonstration of logical branches 2 points – README with architectural explanation